

WAREHOUSE SKU AND QR CODE MANAGER

AI Overview and General Workflow

Document date: 2026-04-01

Application root: d:\Wareehouse Bar Code generator

Primary implementation files: src/mainwindow.cpp, src/mainwindow_history.cpp, src/printsettingsdialog.cpp, src/mainwindow.h

1. What This Software Does

This software is a Windows desktop application built with Qt Widgets and C++. It manages warehouse SKU master data, generates QR-coded serial labels, tracks historical QR records, exports operational reports, and protects sensitive actions with user roles, audit logs, and backup workflows.

The application is intended for warehouse and inventory teams that need one tool for managing master data and outgoing or incoming QR label generation. The software is not a web application. It is a local desktop app backed by an SQLite database.

2. Core Business Purpose

The software solves two connected problems.

First, it maintains a structured product catalog where each item gets a computed SKU based on category rules.

Second, it generates unique QR label values for those SKUs so the warehouse can print stickers, track serial creation over time, and review or correct historical serials later.

3. Main Functional Areas

Dashboard: shows top-level counts such as total SKUs, total generated serials, current quarter volume, prefix counts, and latest SKU cards.

SKU Generation: creates and maintains the SKU master catalog. Users can search records, add new rows, delete the form from existing rows, update fields, delete SKUs, and attach product images.

QR Code Generation: creates one or more QR serials for a selected SKU, shows the generated values in a table, previews a QR image, and exports sticker labels to PDF.

QR Code Manager: loads historical QR rows by SKU and serial filter, displays details, allows controlled edit and delete actions, and can export selected history rows to sticker labels.

Security and Administration: manages user accounts, roles, permission overrides, login/logout switch-user flow, and audit logging.

Database and Backup Operations: loads an existing database, exports a database copy, and manages backups.

backups, runs automated encrypted backups, and keeps retention history.

4. Main Data Entities

`sku_rules`: the category and sub-category rule table used to build valid SKUs.

`sku_catalog`: the SKU master table that stores item identity and warehouse metadata: part name, part number, dimensions, weight, storage, rack, bin, family, image, and

`barcode_log`: the generated QR history table. Each row stores SKU, year, quarter, serial number, barcode value, and timestamps. Deletes are soft deletes.

`barcode_serials`: a tracker table that stores the last used serial for each SKU and combination.

`app_users` and `app_roles`: authentication and authorization tables for login, role permissions, and optional per-user permission override.

`app_audit_log`: records sensitive actions, denials, edits, deletes, and other traceable events.

`app_meta` and `app_backup_history`: store application metadata and automated backup history.

5. Startup Workflow

1. The application starts in `src/main.cpp` and opens `MainWindow`.
2. `MainWindow` builds the UI, menu actions, and signal wiring.
3. The app selects or opens the SQLite database path.
4. If the database is available, the app ensures schema, default roles, seed data, and embedded admin availability.
5. The app prompts for login.
6. After successful login, it loads category rules, SKU lists, history filters, dashboard metrics, and backup scheduling.

If startup database selection is skipped, the window can remain in a no-database state until the user loads a database later.

6. SKU Workflow

The SKU workflow begins in the SKU Generation tab.

Users choose a category and sub-category that come from the seeded SKU rules. They enter item details such as part name, part number, weight, storage information, rack location, product family, comments, and optionally an image.

The app computes the SKU from business rule components:
category digit + sub-category digit + item serial padded to 3 digits + variation code

Variation codes use digits first and then letters for higher values. The app blocks or duplicate SKU creation.

The user can save a new SKU record, update an existing one, or soft-delete a selected one. Deleting a SKU also soft-deletes related barcode records and requires elevated authorization.

7. QR Generation Workflow

The QR workflow begins after a valid SKU exists in the catalog.

The user selects a SKU, chooses a branding prefix, chooses quarter and year, and enters a quantity. The supported prefixes in the current code are SD, SK, and SM.

For each requested serial, the app computes a QR value using:
prefix + SKU + quarter + 2-digit year + serial padded to 5 digits

The generation flow inserts rows into `barcode_log`, refreshes the next-serial tracker in the generated table, and updates dashboard totals. The QR image preview is rendered from the generated value.

This means the software is both a catalog manager and a serial label generator. The SKU defines the product identity. The QR value defines a unique label instance for that product in a given period.

8. Sticker PDF Workflow

After QR rows are generated, the user can export sticker labels to PDF.

The print path uses Qt PrintSupport and a configurable sticker layout. The print setup dialog controls label size, QR size, margins, logo size, and element placement. The routine combines product details, rack and bin details, branding logo, website text, and QR image onto each sticker label.

The same PDF export path can also be used from the history screen for selected historical serial rows.

9. QR History Workflow

The QR Code Manager tab is the operational review screen for generated serials.

The user filters by SKU and optionally by serial text, then loads matching rows. The table shows SKU, part name, serial, generated timestamp, QR value, quarter, and year.

From this screen, authorized users can:
review details for an existing QR record
export selected rows to sticker PDF
edit serial, quarter, year, and date for a selected row
delete selected rows through soft delete

Edit and delete actions require permission checks and admin re-authorization. After a change, the app refreshes serial tracking so future generation continues from the current state.

next value.

10. Security Model

The application requires login for normal use. Passwords are stored as salted SHA-2 hashes.

Base access levels are View Only, Add Data, and Full Access. The database also supports named roles and optional user-specific permission overrides.

Important permissions include add, edit, delete SKU, serial edit, serial delete, print, export, backup or restore, and manage users.

Sensitive actions such as deleting SKUs, deleting serial rows, and editing historical rows require a second admin authorization step with a reason comment. The software logs these actions to the audit table.

11. Export and Backup Workflow

The software supports several output flows.

CSV export for SKU master data.

CSV export for QR summary totals.

Spreadsheet export for all generated serials.

Database export or save-copy workflows.

Automated encrypted backups with retention pruning and backup history tracking.

These flows make the application useful not only for daily operations but also for reporting, traceability, and recovery.

12. Runtime Files and Inputs

Important seeded inputs:

Assets/sku_rules.csv for category and sub-category rules.

Assets/dont open/EXISTING sku DATA.csv or related catalog seed paths for first-time import when available.

Important runtime outputs:

SQLite database, commonly stored as sku.db.

Image files under the application data area when needed.

Monthly runtime logs under the local app data log folder.

Backup files under the configured backup root.

PDF, CSV, and spreadsheet exports chosen by the operator.

13. Mental Model For An AI Assistant

An AI assistant should think of this software as a single-user or small-team desktop application with four linked concerns:

- product master data
- serial label generation
- security and audit
- backup and export operations

The central controller is `MainWindow`. Most business logic, UI wiring, SQL access, and workflow orchestration live there. History-specific behavior is split into `src/mainwindow_history.cpp`. Sticker layout behavior lives in `src/printsettingsdialog.cpp`.

The most important invariant is that SKU master data and QR serial history stay consistent. Changes in one area often require refreshes in lists, dashboard metrics, and next-step tracking.

14. Short Summary

In plain terms, this software helps a warehouse team create standardized SKUs, generate branded QR sticker labels for those SKUs, keep a searchable history of every generated serial, export labels and reports, and control sensitive actions through login, permissions, audit, and backups.